FastFeast Data Pipeline

Windows Setup & Run Guide

For Windows 10 / 11 • Python 3.10+ • PostgreSQL 14+

📋 Read every step fully before running any command. Do not skip steps — each one depends on the previous.

1. Prerequisites

Install these tools before doing anything else. Tick each one off as you go.

Tool	Version Needed	Download
Python	3.10 or higher	python.org/downloads (check 'Add to PATH')
PostgreSQL	14 or higher	postgresql.org/download/windows
Git (optional)	Any recent version	git-scm.com/download/win

Verify Python is installed — open Command Prompt or PowerShell and run:

```
python --version
```

You should see: Python 3.11.x (or higher)

Verify PostgreSQL is installed:

```
psql --version
```

You should see: psql (PostgreSQL) 16.x (or higher)

---

2. Project Setup

Step 1 — Unzip the project

Unzip fastfeast_pipeline.zip to a folder, for example:

```
C:\Projects\fastfeast_pipeline
```

You should see this structure inside:

```
fastfeast_pipeline\
  config\
  src\
  pipeline\
  scripts\
  main.py
```

```
requirements.txt
schema.sql
.env.example
```

Step 2 — Open PowerShell in the project folder

Hold Shift and right-click inside the project folder, then choose 'Open PowerShell window here'. All commands from this point run inside this window.

Step 3 — Create a virtual environment

```
python -m venv venv
venv\Scripts\activate
```

Your prompt will now start with (venv) — this means the environment is active. You must see this before running any other command.

> ⚠ If you close PowerShell and reopen it, you must run venv\Scripts\activate again before running anything.

Step 4 — Install Python dependencies

```
pip install -r requirements.txt
```

This installs: pandas, psycopg2-binary, pyyaml, python-dotenv, reportlab. It may take 1–2 minutes.

Step 5 — Copy the simulation scripts

Copy all 6 provided .py scripts into the scripts\ folder:

- generate_master_data.py
- generate_batch_data.py
- generate_stream_data.py
- add_new_customers.py
- add_new_drivers.py
- simulate_day.py

Verify they are there:

```
dir scripts\
```

---

3. Database Setup

Step 6 — Create the database

Open a second PowerShell window and connect to PostgreSQL:

```
psql -U postgres
```

Enter your postgres password when prompted. Then run these commands inside psql:

```
CREATE DATABASE fastfeast_dw;
CREATE USER pipeline_user WITH PASSWORD 'fastfeast123';
GRANT ALL PRIVILEGES ON DATABASE fastfeast_dw TO pipeline_user;
\q
```

Step 7 — Create the .env file

In your (venv) PowerShell window, create the credentials file:

```
echo DB_PASSWORD=fastfeast123> .env
echo ALERT_EMAIL=>> .env
echo ALERT_PASSWORD=>> .env
```

> Leave ALERT_EMAIL and ALERT_PASSWORD empty for now — email alerts are optional and can be configured later.

Step 8 — Configure pipeline_config.yaml

Open `config\pipeline_config.yaml` in any text editor and confirm the database section looks like this:

```
database:
  host: localhost
  port: 5432
  name: fastfeast_dw
  user: pipeline_user
  password: ${DB_PASSWORD}
```

Also set alerts to disabled:

```
alerts:
  enabled: false
```

Step 9 — Verify the connection

Back in your (venv) window:

```
python main.py --check
```

Expected output:

```
Testing database connection ...
[OK] Database reachable.
Verifying schema ...
[OK] All warehouse tables created / verified.
```

> If you see a connection error, double-check: PostgreSQL is running, DB_PASSWORD in .env is correct, and pipeline_config.yaml has the right host/port/user.

---

4. Running the Pipeline

Step 10 — Generate master data (run once only)

This creates the base source tables — customers, restaurants, drivers, agents, and all lookup tables:

```
python scripts\generate_master_data.py
```

Expected: cities (3), regions (18), customers (510), restaurants (58), drivers (101), agents (30).

> Run this only once. Running it again will overwrite the master data and break the ID sequence for orphan simulation.

Step 11 — Generate batch data for your date

Replace `2026-02-22` with the date you want to simulate:

```
python scripts\generate_batch_data.py --date 2026-02-22
```

This creates `data\input\batch\2026-02-22\` with 13 files (11 CSV + 2 JSON).

Step 12 — Run batch ingestion

```
python main.py --date 2026-02-22 --batch-only
```

Expected output:

```
Batch complete: {'files_attempted': 13, 'files_succeeded': 13, 'files_skipped': 0, ...}
```

This loads all 12 dimension tables into PostgreSQL. A few rows will be rejected — this is normal (intentional data quality issues in the source data).

Step 13 — Generate stream data (simulate orders arriving)

First add some new customers and drivers to create realistic orphan references:

```
python scripts\add_new_customers.py --count 5
python scripts\add_new_drivers.py --count 3
```

Then generate hourly transaction files:

```
python scripts\generate_stream_data.py --date 2026-02-22 --hour 9
python scripts\generate_stream_data.py --date 2026-02-22 --hour 12
python scripts\generate_stream_data.py --date 2026-02-22 --hour 18
```

Each command creates data\input\stream\2026-02-22\HH\ with orders.json, tickets.csv, ticket_events.json.

Step 14 — Run the full pipeline

Open TWO PowerShell windows — both with venv\Scripts\activate active.

Window 1 — start the pipeline:

```
python main.py --date 2026-02-22
```

Window 2 — optionally keep adding more stream data while it runs:

```
python scripts\generate_stream_data.py --date 2026-02-22 --hour 20
```

The pipeline runs silently. All output goes to logs\pipeline.log. Wait 30–60 seconds then press Ctrl+C in Window 1 to stop it.

Step 15 — Verify results

Check fact table row counts:

```
psql -U pipeline_user -d fastfeast_dw -c "SELECT 'fact_order' AS tbl, COUNT(*) FROM
fact_order UNION ALL SELECT 'fact_ticket', COUNT(*) FROM fact_ticket UNION ALL SELECT
'fact_ticket_event', COUNT(*) FROM fact_ticket_event;"
```

Check SLA metrics:

```
psql -U pipeline_user -d fastfeast_dw -c "SELECT * FROM v_sla_daily;"
```

Check quality log:

```
psql -U pipeline_user -d fastfeast_dw -c "SELECT table_name, total_records,
valid_records, rejected_records, status FROM quality_log ORDER BY run_id;"
```

Check quarantined rows:

```
dir data\quarantine\2026-02-22\
```

---

5. Simulate a Full Day (Quick Method)

Instead of running each script manually, use simulate_day.py to run everything in the correct order automatically:

Window 1 — start the pipeline first:

```
python main.py --date 2026-02-22
```

Window 2 — run the full day simulation:

```
python scripts\simulate_day.py --date 2026-02-22 --skip-master
```

This automatically: generates batch data, adds random new customers/drivers, and generates stream data for 8 hourly slots.

---

6. Troubleshooting

Problem	Solution
psql not recognized	Add PostgreSQL bin folder to PATH: C:\Program Files\PostgreSQL\16\bin
pip install fails on psycopg2	Use psycopg2-binary — it is already in requirements.txt. If still failing, install Visual C++ Build Tools.
Connection refused on port 5432	Open Services (Win+R → services.msc) and start 'postgresql-x64-16'.
(venv) disappears from prompt	Run venv\Scripts\activate again. Must be done every time you open a new terminal.
No output from main.py	This is correct — all output goes to logs\pipeline.log. Open it to see progress.
Batch file: files_skipped > 0	The file was already processed. Delete file_tracker.db and re-run to reprocess all files.
Want to start completely fresh	Delete file_tracker.db, truncate all warehouse tables with CASCADE, then re-run from Step 10.

---

7. Quick Command Reference

Command	What it does
python main.py --check	Test DB connection + verify schema
python main.py --date YYYY-MM-DD --batch-only	Load batch dimension files only
python main.py --date YYYY-MM-DD	Full pipeline (batch + stream watcher)
python scripts\generate_master_data.py	Create base source tables (once only)
python scripts\generate_batch_data.py --date DATE	Create daily batch snapshot

python scripts\generate_stream_data.py --date DATE --hour HH	Create one hourly stream slot
python scripts\add_new_customers.py --count N	Add N new customers (creates orphans)
python scripts\add_new_drivers.py --count N	Add N new drivers (creates orphans)
python scripts\simulate_day.py --date DATE --skip-master	Simulate a full day automatically

Pipeline logs are at logs\pipeline.log. Rejected rows are at data\quarantine\YYYY-MM-DD\